

RAG Practice Knowledge Base

Artificial Intelligence, LLMs, Retrieval-Augmented Generation, Embeddings, and Vector Search

1. Introduction to Artificial Intelligence

Artificial Intelligence (AI) is a field of computer science focused on building systems that can perform tasks that normally require human intelligence. These tasks include understanding language, recognizing images, learning from data, making predictions, solving problems, and supporting decisions. Modern AI applications are used in search engines, recommendation systems, fraud detection, healthcare, education, transportation, finance, and software development.

2. Machine Learning

Machine Learning is a major area of AI in which computers learn patterns from data instead of being explicitly programmed for every possible situation. Supervised learning uses labeled examples, unsupervised learning discovers patterns in unlabeled data, and reinforcement learning learns through interaction with an environment and feedback. Common machine learning algorithms include linear regression, logistic regression, decision trees, random forests, support vector machines, and neural networks.

3. Deep Learning

Deep Learning uses neural networks with multiple layers to learn complex representations from data. Convolutional Neural Networks are commonly used for images, while recurrent architectures and transformer-based models are widely used for sequential and language tasks. Deep learning generally requires substantial training data and computational resources, but it can achieve strong performance on difficult perception and language problems.

4. Natural Language Processing

Natural Language Processing (NLP) focuses on enabling computers to process and understand human language. Common NLP tasks include text classification, sentiment analysis, named entity recognition, translation, summarization, question answering, and information extraction. Modern NLP systems often use transformer architectures because transformers can model relationships between words across long contexts.

5. Large Language Models

Large Language Models, or LLMs, are neural network models trained on large collections of text. They learn statistical and semantic patterns that allow them to generate and transform language. LLMs can answer questions, summarize documents, write code, translate text, classify information, and assist with research. However, an LLM can produce inaccurate information, especially when the required knowledge is missing, ambiguous, private, or outside its reliable context.

6. Generative AI

Generative AI refers to systems capable of producing new content. Depending on the model, the output may be text, images, audio, video, code, or structured data. A user typically provides a prompt that

describes the desired task. The model processes the prompt and generates an output based on patterns learned during training and the context supplied at inference time.

7. Retrieval-Augmented Generation

Retrieval-Augmented Generation, commonly called RAG, combines information retrieval with language generation. Instead of asking a language model to answer a question using only its internal learned parameters, a RAG application first searches an external knowledge source. Relevant information is retrieved and included in the prompt given to the language model. This approach is especially useful for private documents, company knowledge bases, manuals, research collections, and frequently changing information.

8. RAG Pipeline

A basic RAG pipeline consists of document ingestion, document splitting, embedding generation, vector storage, retrieval, prompt construction, and answer generation. During ingestion, files are loaded from sources such as PDFs, TXT files, CSV files, websites, or databases. Documents are divided into chunks. Each chunk is converted into an embedding and stored in a vector database. When a user asks a question, the question is embedded and compared with stored embeddings. The most relevant chunks are retrieved and supplied to the language model as context.

9. Document Loaders

Document loaders convert different sources into a format that an application can process. A text loader can read TXT files, PDF loaders can extract content from PDF documents, CSV loaders can process tabular data, and web loaders can collect information from websites. In frameworks such as LangChain, document loaders provide a convenient abstraction for handling many data sources. The quality of extracted text matters because missing or incorrectly ordered content can reduce retrieval quality.

10. Text Splitting and Chunking

Large documents are normally divided into smaller chunks before embedding. Chunking helps retrieval systems return focused sections rather than entire documents. A chunk that is too small may lose important context, while a chunk that is too large may contain unrelated information. Chunk size and overlap should be selected according to the document type and the expected questions. Recursive character splitting is one common approach for general text.

11. Embeddings

An embedding is a numerical vector representing the semantic characteristics of a piece of information. Similar meanings tend to have vectors that are close together according to a chosen similarity metric. Embeddings are useful for semantic search because a query can retrieve relevant information even when the query does not use exactly the same words as the source document. Common similarity measures include cosine similarity and Euclidean distance.

12. Vector Databases

Vector databases are designed to store and search high-dimensional vectors efficiently. Examples include FAISS, Chroma, Pinecone, Weaviate, Milvus, and other vector search systems. A typical record contains an embedding along with metadata and the original text or a reference to it. During retrieval, the system searches for vectors that are most similar to the query embedding.

13. Retrieval

Retrieval is the process of finding relevant information for a user query. Similarity search is a common retrieval technique. More advanced RAG systems may combine keyword search with semantic search, use metadata filters, or apply a reranking model after initial retrieval. Retrieval quality is critical because a language model cannot reliably use information that was never provided to it.

14. Prompt Construction

After relevant chunks are retrieved, the application constructs a prompt containing the user's question and the retrieved context. A good prompt can instruct the model to answer using only the supplied context, explain uncertainty when information is missing, and avoid inventing facts. Clear instructions can make the final response more grounded and useful.

15. Hallucination and Grounding

A hallucination occurs when an AI model generates information that is unsupported, incorrect, or fabricated. RAG can reduce hallucinations by providing relevant source material, but it does not guarantee correctness. Developers should evaluate retrieval accuracy, inspect source documents, use appropriate prompts, and introduce validation mechanisms for high-impact applications.

16. RAG Evaluation

RAG systems can be evaluated at multiple levels. Retrieval evaluation checks whether the correct documents or chunks were retrieved. Generation evaluation checks whether the final answer is correct, relevant, complete, and supported by the retrieved context. Useful evaluation questions include whether the answer contains unsupported claims, whether important information was missed, and whether the retrieved context actually answers the user's question.

17. Metadata

Metadata is additional information associated with a document or chunk. Examples include document title, author, category, date, source, department, product, and page number. Metadata filtering can improve retrieval by limiting the search to relevant categories. For example, a company knowledge system may retrieve only documents belonging to the Human Resources department when answering an employee policy question.

18. RAG with CSV Data

CSV files contain rows and columns and are useful for structured information. A RAG application can load CSV records and convert rows into searchable documents. Each row may represent a product, employee policy, support article, course, customer record, or research item. Metadata such as category and identifier can be preserved to make filtering and source tracking easier.

19. RAG with PDFs

PDF files are common sources for RAG because organizations store manuals, reports, research papers, policy documents, and technical documentation as PDFs. A PDF loader extracts text, which can then be cleaned and split into chunks. Some PDFs contain tables, scanned images, or complicated layouts, so extraction quality should be checked before building embeddings.

20. Practical RAG Project

A beginner RAG project can use a PDF, TXT file, and CSV file as the knowledge base. The application loads all documents, splits them into chunks, creates embeddings, and stores the vectors in a local vector database. A user can then type a question such as 'What is Retrieval-Augmented Generation?' or 'Why is chunking important?' The system retrieves relevant information and generates a grounded answer.

21. Example Questions

Example questions for testing include: What is artificial intelligence? What is machine learning? How is deep learning different from traditional machine learning? What is an LLM? What is RAG? Why do RAG applications use embeddings? What is a vector database? Why are documents split into chunks? What is cosine similarity? What is metadata? How can RAG reduce hallucinations? What are the stages of a RAG pipeline? How can CSV files be used in RAG? Why can PDF extraction be difficult? How should a RAG application be evaluated?

22. Advanced RAG

Advanced RAG systems can include query rewriting, multi-query retrieval, hybrid search, reranking, parent-document retrieval, contextual compression, metadata filtering, and agentic retrieval. Query rewriting changes a user's question into one or more retrieval-friendly queries. Reranking evaluates initially retrieved documents and places the most useful results first. These techniques can improve performance on complex questions.

23. RAG and Agents

AI agents can use RAG as one of several tools. An agent may decide when to search a knowledge base, when to call an API, when to query a database, or when to perform another action. In an agentic system, retrieval is not necessarily a fixed first step. The agent can reason about which source or tool is appropriate and use retrieved information to continue the task.

24. Security and Privacy

RAG applications may process sensitive company or user information, so access control and data security are important. The application should restrict retrieval according to user permissions. Private documents should not be exposed to unauthorized users. Logs should be handled carefully, and developers should consider how documents, prompts, retrieved context, and generated responses are stored.

25. Conclusion

Retrieval-Augmented Generation is an important technique for connecting language models with external knowledge. A strong RAG system requires more than simply adding a vector database. Document quality, chunking, embeddings, retrieval, metadata, prompt design, evaluation, and security all influence the final result. Practicing with TXT, PDF, and CSV files provides a useful foundation for understanding how real-world RAG applications are built.